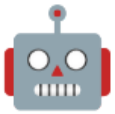


Agentic AI Pipeline for Automated EDA — Project Plan

🎯 Build a multi-agent system that takes a raw dataset (CSV / Excel / DB table) and autonomously plans, executes, and narrates a full Exploratory Data Analysis — producing cleaned data, visualizations, statistical findings, and a written report with minimal human input.

1. Problem & Goal

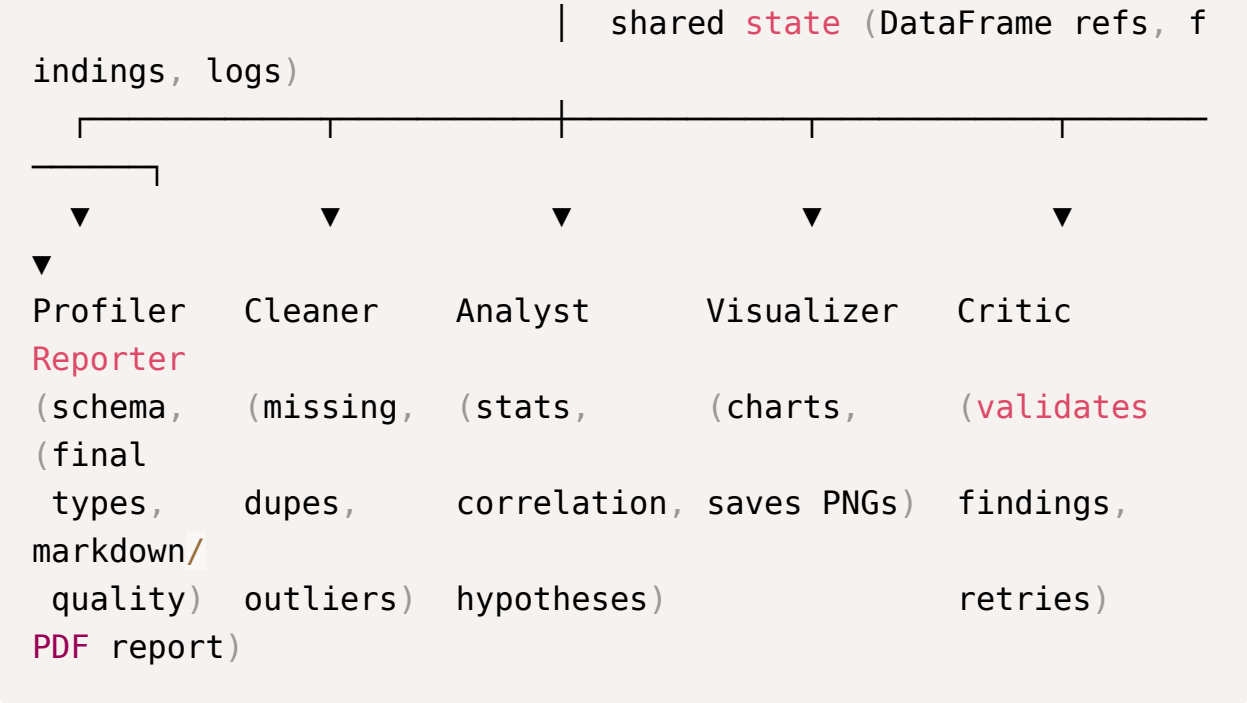
Manual EDA is repetitive: load data, profile it, clean it, plot distributions, check correlations, spot outliers, and write up findings. This project automates that loop with a team of specialized LLM-driven agents that reason about *what to analyze next* and generate + execute the code to do it.

Success criteria

- [ ] Accepts any tabular dataset and runs end-to-end without manual coding
- [ ] Produces a reproducible notebook/script + a human-readable report
- [ ] Self-corrects when generated code errors out
- [ ] Lets a human steer via natural-language instructions ("focus on churn drivers")

2. High-Level Architecture

```
┌──────────────────────────────────────────────────┐
│              Orchestrator (LangGraph)            │
│   plans steps, routes to agents, tracks state    │
└──────────────────┬───────────────────────────────┘
```



A **code-execution sandbox** sits underneath: agents emit Python, it runs in an isolated kernel, results (tables, images, errors) flow back into state.

3. The Agents

Agent	Responsibility	Key tools
Orchestrator	Decides the next EDA step based on current state & user intent	LangGraph state machine
Profiler	Infers schema, dtypes, cardinality, missingness, basic stats	`pandas`, `ydata-profiling`
Cleaner	Proposes + applies imputation, dedup, type fixes, outlier handling	`pandas`, `scikit-learn`
Analyst	Univariate/bivariate stats, correlations, hypothesis generation	`scipy`, `statsmodels`
Visualizer	Chooses + generates appropriate charts, saves artifacts	`matplotlib`, `seaborn`, `plotly`
Critic / Validator	Reviews generated code & findings, triggers retries on error or low quality	LLM self-review loop
Reporter	Synthesizes findings into a narrative report	Markdown/PDF export

4. Tech Stack

- **Orchestration:** LangGraph (stateful multi-agent graph) — plays to your strengths
- **LLM layer: Groq API** (free tier, OpenAI-compatible, very fast inference) via `langchain-groq`'s `ChatGroq`; structured outputs (Pydantic) through tool/function calling
  - Recommended models: `llama-3.3-70b-versatile` for the reasoning-heavy Orchestrator/Analyst/Critic; `llama-3.1-8b-instant` for cheap/fast steps like the Profiler
  - Keep the LLM behind a thin wrapper so you can swap providers later without touching agent logic
- **Execution:** Jupyter kernel or a restricted `exec` sandbox (consider `e2b` / `nbclient` for isolation)
- **Data:** `pandas`, `polars` (optional for scale), `pyarrow`
- **Viz:** `matplotlib`, `seaborn`, `plotly`
- **Backend/API:** FastAPI
- **Frontend (optional):** React or Streamlit for upload + live report view
- **Infra:** Docker, `.env` config, CI/CD (GitHub Actions) — mirrors your Job-Hunt-Agent setup

⚡ **Working with Groq (free tier)**

• Set `GROQ_API_KEY` in `.env`; the client is OpenAI-compatible, so most LangChain tooling works unchanged.

• Free tier has **rate limits** (requests + tokens per minute) — add retry-with-backoff and cap the graph's iterations so the Critic loop can't burn through quota.

• Groq serves **open models only** (Llama, Mixtral, Gemma, Qwen, etc.) — no GPT/Claude. Prefer the 70B model where reasoning quality matters, and only use tool-calling-capable models for structured outputs.

• No image input on most models, so keep the Visualizer generating code (charts) rather than interpreting images.

5. Pipeline Stages

1. **Ingest** — load file/DB, detect delimiter/encoding, sample large data
2. **Profile** — generate a data-quality snapshot and pass to orchestrator
3. **Plan** — orchestrator drafts an ordered EDA task list from profile + user goal
4. **Clean** — apply and log transformations (keep before/after diffs)
5. **Analyze** — run stats, correlations, group-bys, hypothesis checks
6. **Visualize** — generate + save charts, embed references in state
7. **Critique loop** — validator checks each artifact; re-runs failed/weak steps
8. **Report** — assemble narrative + charts + reproducible code

6. Suggested Repo Structure

```
eda-agent/
├── app/
│   ├── graph/            # LangGraph nodes + edges
│   │   ├── orchestrator.py
│   │   ├── agents/       # profiler, cleaner, analyst, ...
│   │   └── state.py      # shared TypedDict/Pydantic state
│   ├── tools/            # code executor, chart saver, loade
```

```
rs
│   ├── prompts/          # per-agent system prompts
│   └── api/              # FastAPI routes
├── sandbox/              # isolated execution runtime
├── reports/              # generated outputs
├── tests/
├── Dockerfile
└── requirements.txt
```

7. Milestones

☐ **Phase 1 — Core loop (Week 1–2):** single Analyst agent + code executor that profiles a CSV and returns summary stats

☐ **Phase 2 — Multi-agent graph (Week 2–3):** add Profiler, Cleaner, Visualizer nodes wired in LangGraph with shared state

☐ **Phase 3 — Self-correction (Week 3–4):** Critic agent + retry-on-error, structured logging

☐ **Phase 4 — Reporting (Week 4–5):** Markdown/PDF report generation with embedded charts

☐ **Phase 5 — Interface + deploy (Week 5–6):** FastAPI + Streamlit/React upload UI, Dockerize, CI/CD

8. Evaluation & Responsible AI

- **Correctness:** compare agent findings against a hand-done EDA on 2–3 benchmark datasets (Titanic, Adult Income, a churn dataset)
- **Reproducibility:** the emitted notebook must re-run and reproduce every artifact
- **Reliability:** track code-error rate and self-correction success rate
- **Safety/fairness:** flag sensitive attributes (gender, race, age) and surface potential bias in distributions — ties into your Responsible AI interest

9. Key Risks & Mitigations

- **Arbitrary code execution** → run in a sandboxed kernel, never on the host; whitelist libraries
- **LLM hallucinating column names** → always ground prompts in the live schema from the Profiler
- **Runaway loops / cost** → cap graph iterations, add token/step budgets
- **Large datasets** → sample intelligently, push heavy ops to `polars`

10. Stretch Goals

- Natural-language follow-up Q&A over the analyzed dataset (RAG over findings)
- Auto-suggest ML modeling steps after EDA
- Support for multi-table / relational EDA with join inference
- Indic-language report output (aligns with your civic-sector NLP interest)